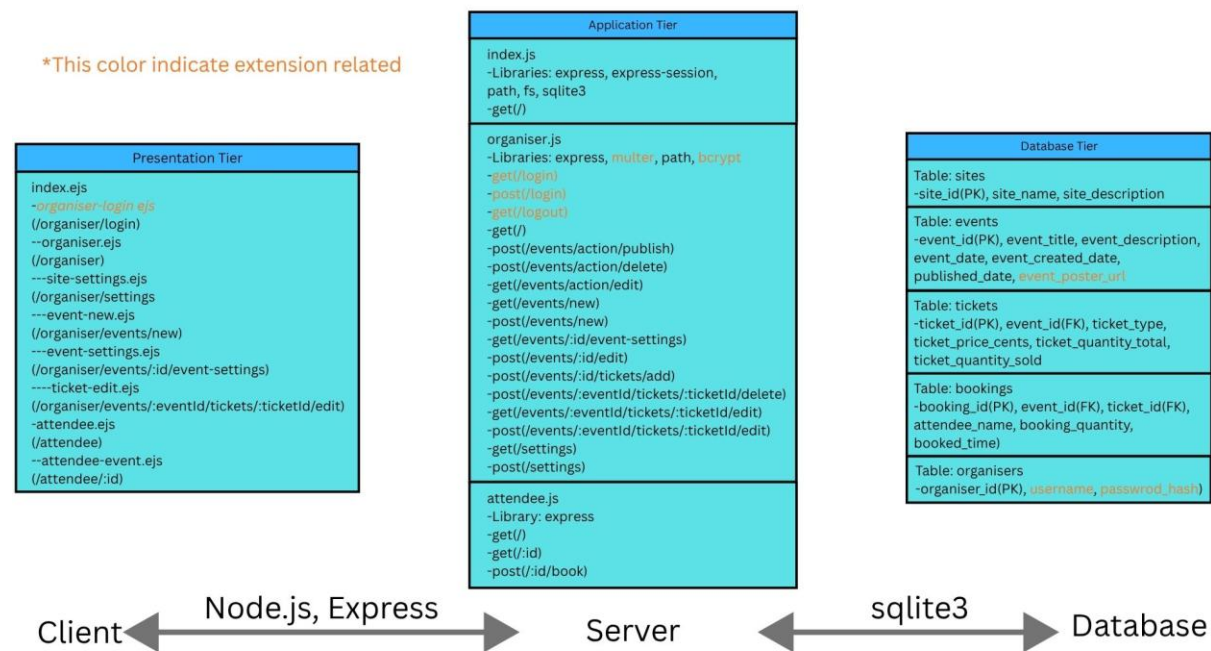
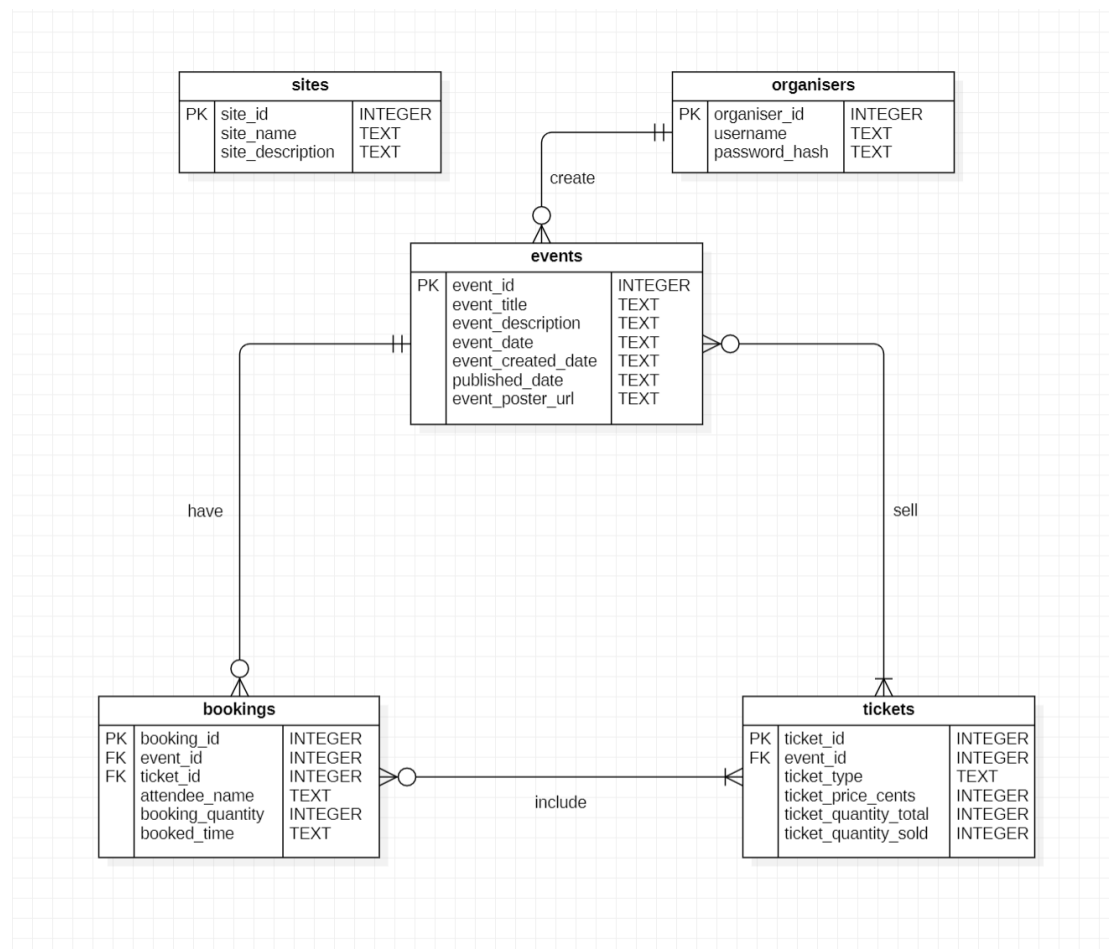


Schematic Diagram



This is the three tiers schematic diagram of my event management. The color highlighted are the part of extensions. I have created organiser-login just for the organiser side authentication process since we shouldn't let all the users to have access to organiser page.

Entity Relationships Diagram (ERD)



event_poster_url, username and password_hash are the extension related attributes in my database.

Extension 1: Photo Upload

One of my extensions is to upload the image to the server side storage. I added this because it would be quite dry in event page without the image and I also don't want hard coded one. To achieve this, I found that I can use multer library.

Multer is a popular middleware for handling multipart/form-data, which is the encoding used for file uploads in HTML forms. In my projects, multer allow the user/organiser to upload the desire image to the server.

How the photo upload works

User will first submit the form by either creating or editing the event together with the image file from the local storage. Multer will processes the file upload and save it to a folder, in my case it

is /public/uploads directory. The file name of the uploaded image will then be saved in the database under event_poster_url column. For displaying the image, the program will get the filename from database and generate in image tag.

Implementation

First I use npm install multer to install

Multer Setup (organiser.js, Line 9-18)

```
destination: function (req, file, cb) {  
  cb(null, './public/uploads/');  
},
```

The above code specifies the directory to upload the image

```
filename: function (req, file, cb) {  
  cb(null, 'event-' + Date.now() + path.extname(file.originalname));  
}
```

The above code is for giving file name + the file extension

Handling the uploaded file (organiser.js, Line 119-135)

```
router.post('/events/new', upload.single('event_poster'), (req, res, next) =>  
{  
  let posterFile = req.file ? req.file.filename : null;  
  
  const eventQuery = `  
    INSERT INTO events (event_title, event_description, event_date,  
event_poster_url)  
    VALUES (?, ?, ?, ?)  
  `;  
  const eventParams = [event_title, event_description, event_date,  
posterFile];  
});
```

upload.single('event_poster') tells Multer to catch a single file named event_poster from the form. req.file is an object added by Multer if a file was uploaded with the form. req.file.filename is the generated file name that Multer saved for the uploaded file. So if there is no uploaded file, posterFile will be null and there will be no photo for that event.

Image Upload (event-new.ejs, Line 53-54)

```
<label for="event_poster">Upload Poster Image</label>  
  <input type="file" id="event_poster" name="event_poster"  
accept="image/*">
```

Image Upload (event-settings.ejs, Line 36 – 41)

```
<label for="event_poster">Upload Poster Image</label>
<input type="file" id="event_poster" name="event_poster"
accept="image/*">
  <input type="hidden" name="existing_poster_url" value="<%=
event.event_poster_url %>">
  <% if (event.event_poster_url) { %>
    
  <% } %>
```

if (event.event_poster_url) check whether the image exist or not in the storage

Image Display (attendee-event.ejs, Line 17-21)

```
<% if (event.event_poster_url) { %>
  
<% } %>
```

If there is corresponding event poster for the selected event, that poster/image will be print as an image tag.

Highlights of the extension 1

- The events table includes an event_poster_url field to store the uploaded filename
- Multer is configured to store the file with the current timestamp so the file name will always be unique
- The attendee page and event page check if an image exists for each event and will still work even if there is no image file

Extension 2: Organiser Authentication

For this extension, I had used bcrypt library to hash the passwords and store in database. bcrypt is a popular Node.js library used to hash passwords securely. It uses a computationally intensive algorithm, making it difficult for attackers to reverse or brute-force hashed passwords.

This extension implements an authentication system that requires organisers to log in before accessing the management system.

How the hashing work

Bcrypt will take the password and the salt to hash and return the result which is hard for the attacker to guess even with powerful computation

Implementation

First I install bcrypt and create a organisers table in database to store user name and password. Right now, the user is just me so there will only be one entry in this table but in future we still can use this for storing multiple details.

Organisers Table (db_schema.sql, Line 45-49)

```
CREATE TABLE IF NOT EXISTS organisers (  
  organiser_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE,  
  password_hash TEXT  
);
```

Hashing Password (run in Node)

```
require('bcrypt').hash('iamgood', 10).then(console.log)
```

This will give the hashed password using salt:10

Authentication (organiser.js, Line 21-27)

```
function requireLogin(req, res, next) {  
  if (!req.session.organiserId && req.path !== '/login') {  
    return res.redirect('/organiser/login');  
  }  
  next();  
}  
router.use(requireLogin);
```

Login Route (organiser.js, Line 32-52)

```
router.get('/login', (req, res) => {  
  res.render('organiser-login', { error: null });  
});  
  
router.post('/login', (req, res) => {  
  const { username, password } = req.body;
```

```

    global.db.get('SELECT * FROM organisers WHERE username = ?', [username],
(err, user) => {
    if (err) return res.render('organiser-login', { error: "Database error."
});
    if (!user) return res.render('organiser-login', { error: "Invalid username
or password." });

    bcrypt.compare(password, user.password_hash, (err, valid) => {
    if (valid) {
        req.session.organiserId = user.organiser_id;
        res.redirect('/organiser');
    } else {
        res.render('organiser-login', { error: "Invalid username or password."
});
    }
    });
});
});
});

```

Bcrypt compare the input password from the user with the hash value from database. If match, load the organiser page. If not, inform user about wrong username or password.

Logout Route (organiser.js, line 55-57)

```

router.get('/logout', (req, res) => {
    req.session.destroy(() => res.redirect('/'));
});

```

Highlight of Extension 2

- The stored password is hashed value instead of plain text so it is not easy to get access
- All the admin stuffs are protected by authentication